Dynamic Analysis / Mutation Testing

Software Testing and Quality Assurance

Dr. Karim Akilal

April 13, 2026

Université de Bejaïa

Quand la couverture ne suffit pas!

Vos tests passent, sont-ils assez?

- Vous avez atteint 100% de couverture.
- Toutes vos lignes ont été testées.
- Des bugs apparaissent quand même!

Vos tests passent, sont-ils assez?

- Vous avez atteint 100% de couverture.
- Toutes vos lignes ont été testées.
- Des bugs apparaissent quand même!

La *couverture* vous dit quelles lignes ont été testées (exécutées), et non pas si vos tests détectent des défauts.

Suites de tests valides et complètes

- Une suite de tests est dite **valide** si ses tests **n'échouent** que sur des programmes incorrects, et par conséquent, les erreurs trouvées sont bel et bien réelles

- Une suite de tests est dite **complète** si ces tests **ne réussissent** que sur des programmes corrects. Autrement dit, si le programme contient une erreur, elle est détectée.

Un exemple concret

```c
/* Compute the salary with a bonus */
double bonus(double salary, double rate) {
    return salary + salary * rate;
}

void test_bonus(void) {
    double result = bonus(90000.0, 0.2);
    assert(result != 0.0);   /* OK mais ça ne prouve rien d'important */
}
```

Un exemple concret

```c
/* Compute the salary with a bonus */
double bonus(double salary, double rate) {
    return salary + salary * rate;
}

void test_bonus(void) {
    double result = bonus(90000.0, 0.2);
    assert(result != 0.0);   /* OK mais ça ne prouve rien d'important */
}
```

Changez * en + et le test passe quand même. **Le mutant a survécu.**

Concepts de base

Qu'est ce qu'un mutant?

Un **mutant** est une copie du code avec une petite modification faite exprès.

```
/* Original */
if (x > 0) { ... }

/* Mutant - relational operator replaced */
if (x >= 0) { ... }
```

Toute la suite de tests doit être exécutée avec le mutant. Et celui-ci est dit:

- **Killed** — Au moins un test échoue → défaut détecté (OK)
- **Survived** — Tous les tests passent → La suite l'a raté (Oh NO!)
- **Equivalent** — Le mutant est sémantiquement identique (IGNORE)

Mutation score

$$\text{mutation score} = \frac{\text{killed mutants}}{\text{total mutants} - \text{equivalent mutants}}$$

- **High coverage + low mutation score** $\rightarrow$ assertions faibles.
- But: $> 80\%$ dans les modules dit *critiques*.

Killed vs. survived

```
int is_strictly_positive(int x) { return x > 0; }
```

Test input	Original	Mutant (>=)	Verdict
`x = 1`	1	1	same
`x = 0`	0	**1**	**killed** if asserted
`x = -1`	0	0	same

Killed vs. survived

```
int is_strictly_positive(int x) { return x > 0; }
```

Test input	Original	Mutant (>=)	Verdict
`x = 1`	1	1	same
`x = 0`	0	**1**	**killed** if asserted
`x = -1`	0	0	same

Sans le test pour `x = 0`, le mutant aurait **survécu**.

Mutation Operators

Arithmetic operator replacement (AOR)

```c
/* Original: Euclidean distance squared */
double dist_squared(double x, double y) {
    return x * x + y * y;
}
```

Quelques mutants:

```c
return x * x - y * y;    /* AOR: + → - */
return x * x * y * y;    /* AOR: + → * */
return x + x + y * y;    /* AOR: * → + (inner) */
```

Arithmetic operator replacement (AOR)

```c
/* Original: Euclidean distance squared */
double dist_squared(double x, double y) {
    return x * x + y * y;
}
```

Quelques mutants:

```c
return x * x - y * y;   /* AOR: + → - */
return x * x * y * y;   /* AOR: + → * */
return x + x + y * y;   /* AOR: * → + (inner) */
```

```c
/* Ce test tue les trois mutants : (fabs = float abs)  */
assert(fabs(dist_squared(2.0, 5.0) - 29.0) < 1e-9);
```

Relational operator replacement (ROR)

```c
/* Packet length */
int valid(len) {
    if (len < MIN_SIZE) return ERR_TOO_SMALL;
    if (len > MAX_SIZE) return ERR_TOO_BIG;
    return ERR_GOOD;
}
```

Mutants sur la première condition seule:

```c
if (len <= MIN_SIZE) return ERR_TOO_SMALL;
if (len == MIN_SIZE) return ERR_TOO_SMALL;
if (len != MIN_SIZE) return ERR_TOO_SMALL;
```

Relational operator replacement (ROR)

```c
/* Packet length */
int valid(len) {
    if (len < MIN_SIZE) return ERR_TOO_SMALL;
    if (len > MAX_SIZE) return ERR_TOO_BIG;
    return ERR_GOOD;
}
```

Mutants sur la première condition seule:

```c
if (len <= MIN_SIZE) return ERR_TOO_SMALL;
if (len == MIN_SIZE) return ERR_TOO_SMALL;
if (len != MIN_SIZE) return ERR_TOO_SMALL;
```

Nous avons besoin de tests pour `len == MIN_SIZE` and `len == MAX_SIZE` pour tuer les mutant sur les limites (boundaries)

ROR — Vous souvenez-vous de BVA?

```c
int validate_age(int age) {
    return age >= 18;
}

return age > 18;     /* off-by-one */
return age <= 18;    /* inversé */
return age == 18;    /* Seulement 18 est accepté */
```

ROR — Vous souvenez-vous de BVA?

```c
int validate_age(int age) {
    return age >= 18;
}

return age > 18;     /* off-by-one */
return age <= 18;    /* inversé */
return age == 18;    /* Seulement 18 est accepté */

/* Sans ces tests, age > 18 survit */
assert(validate_age(18) == 1);
assert(validate_age(17) == 0);
assert(validate_age(19) == 1);
```

Logical connector replacement (LCR)

```
int can_withdraw(int is_active, int good_pin, int has_funds) {
    return is_active && good_pin && has_funds;
}
```

Quelques mutants:

```
return is_active || good_pin && has_funds;
return is_active && good_pin || has_funds;
return is_active || good_pin || has_funds;
```

Logical connector replacement (LCR)

```c
int can_withdraw(int is_active, int good_pin, int has_funds) {
    return is_active && good_pin && has_funds;
}
```

Quelques mutants:

```c
return is_active || good_pin && has_funds;
return is_active && good_pin || has_funds;
return is_active || good_pin || has_funds;
```

Nous avons besoin de tests où **exactement un** flag est faux. 3 tests.

```c
assert(can_withdraw(0, 1, 1) == 0);   /* inactive */
assert(can_withdraw(1, 0, 1) == 0);   /* wrong PIN */
assert(can_withdraw(1, 1, 0) == 0);   /* no funds*/
```

And there's more...

Nous pouvons envisager des mutants de plusieurs formes:

- **SVR (Scalar Variable Replacement**: remplacement d'une variable par une autre.
- **UOD - Unary Operator Deletion**: Suppression d'un opérateur unaire (`!` ou `-`)
- **ASR - Assignment Operator Replacement**: Remplacement de (`=`, `+=`, `*=`, etc)
- **AOD** - arithmetic operator deletion
- **BCR** - break continue replacement
- **COD** - conditional operator deletion
- **COI** - conditional operator insertion
- **CRP** - constant replacement
- **DDL** - decorator deletion
- **EHD** - exception handler deletion
- etc.

Outils et workflow

Quelques outils pour le C

- Mull: https://github.com/mull-project/mull
- Dextool Mutate: https://joakim-brannstrom.github.io/dextool/plugin/mutate/
- UniversalMutator: https://github.com/agroce/universalmutator

Exemple: Mull (avec Makefile)

```
# Specify Clang version (Mull is version-specific)
CC = clang-12

# Mull-specific flags
MULL_FLAGS = -fexperimental-new-pass-manager \
             -fpass-plugin=/usr/lib/mull-ir-frontend-12 \
             -g -grecord-command-line

# Add these flags to your compilation rule
all: main.cpp
    $(CC) $(MULL_FLAGS) -O0 main.cpp -o my_program

# Target to run mutation testing
mull-test: my_program
    mull-runner-12 ./my_program
```

Le problème du mutant équivalent

Quelques mutants sont **sémantiquement identiques** — On ne peut pas les tuer.

```
/* Original */
for (int i = 0; i < n; i++) { ... }

/* Mutant A - equivalent: same iteration count */
for (int i = 0; i <= n - 1; i++) { ... }

/* Mutant B - NOT equivalent: off-by-one */
for (int i = 0; i < n - 1; i++) { ... }
```

Le problème du mutant équivalent

Quelques mutants sont **sémantiquement identiques** — On ne peut pas les tuer.

```
/* Original */
for (int i = 0; i < n; i++) { ... }

/* Mutant A - equivalent: same iteration count */
for (int i = 0; i <= n - 1; i++) { ... }

/* Mutant B - NOT equivalent: off-by-one */
for (int i = 0; i < n - 1; i++) { ... }
```

Le mutant A n'est pas *tuable*. Ne vous fatiguez pas.

Qu'est ce qui mérite votre attention?

Des mutants il y en a **énormément**, concentrez-vous sur ceux liés à:

- Logique métier et algorithmes.
- Transitions (automates).
- Gestion correcte de la mémoire (chemins `malloc`/`free`)
- Tout code qui aurait causé des incidents auparavant.